Healthcare Appointment Management System

Complete Feature Breakdown & Implementation Guide

Tech Stack: Next.js · Supabase (PostgreSQL + Auth + Realtime + Storage + Edge Functions)

60	28	32	11
Total Features	Original Spec	✨New Additions	Modules

Feature Index

ID	Feature Name	Type	Module
F01	Email & Password Registration / Login	Original	Auth
F02	✨OTP Phone Verification at Registration	✨Added	Auth
F03	✨Two-Factor Authentication (2FA)	✨Added	Auth
F04	✨Social Login (Google & Apple)	✨Added	Auth
F05	Role-Based Access Control (RBAC)	Original	Auth
F06	Patient Profile Creation & Management	Original	Patient
F07	Family Account Management	Original	Patient
F08	Patient Appointment History & Notes	Original	Patient
F09	✨Document Vault (Upload Prescriptions & Reports)	✨Added	Patient
F10	✨Medical History Form (Allergies, Conditions, Medications)	✨Added	Patient
F11	✨Favourite Doctors & Facilities	✨Added	Patient
F12	✨Patient Home Dashboard	✨Added	Patient
F13	Doctor Profile (Bio, Availability, Fees)	Original	Facility Mgmt
F14	Facility Profile Management	Original	Facility Mgmt
F15	Facility Ratings & Reviews	Original	Facility Mgmt
F16	Multi-Doctor Team Calendar	Original	Facility Mgmt
F17	✨Doctor Live Availability Status	✨Added	Facility Mgmt
F18	✨Review Moderation Panel (Facility)	✨Added	Facility Mgmt
F19	Appointment Booking	Original	Scheduling
F20	Appointment Rescheduling & Cancellation	Original	Scheduling
F21	One-Tap Follow-Up Rebooking	Original	Scheduling
F22	Queue Management	Original	Scheduling
F23	Web Check-In	Original	Scheduling
F24	Calendar Sync (Google / iCal / Outlook)	Original	Scheduling
F25	Overbooking Control (Race Conditions, Buffer, Walk-ins, Emergencies)	Original	Scheduling
F26	✨Waitlist Management	✨Added	Scheduling
F27	✨Emergency / Walk-In Slot Booking	✨Added	Scheduling
F28	Multi-Location Booking	Original	Scheduling
F29	Digital Payment Integration (Thawani / Ompay)	Original	Payments
F30	Earnings Dashboard (Facility)	Original	Payments
F31	✨Invoice & Receipt Generation (PDF)	✨Added	Payments
F32	✨Refund Management	✨Added	Payments
F33	✨Insurance & Coverage Verification	✨Added	Payments
F34	Automated SMS / Email / WhatsApp Reminders	Original	Notifications
F35	Push Notifications (Mobile & Web)	Original	Notifications
F36	✨In-App Notification Centre	✨Added	Notifications
F37	✨Doctor-Patient Secure Messaging	✨Added	Notifications

ID	Feature Name	Type	Module
F38	✨Broadcast Announcements (Facility to Patients)	✨Added	Notifications
F39	AI Doctor Suggestion	Original	AI
F40	In-App Symptom Checker	Original	AI
F41	AI-Based Scheduling Assistant	Original	AI
F42	✨Prescription OCR Scanner	✨Added	AI
F43	✨AI Post-Visit Health Insights	✨Added	AI
F44	✨Video Consultation (Telemedicine)	✨Added	Telehealth
F45	✨Pre-Consultation Form	✨Added	Telehealth
F46	✨Virtual Waiting Room	✨Added	Telehealth
F47	Analytics Dashboard (Booking Trends & Patient Demographics)	Original	Analytics
F48	✨Staff Performance Metrics	✨Added	Analytics
F49	Revenue Reports & Payout History	Original	Analytics
F50	✨PDF Report Export	✨Added	Analytics
F51	✨Prescription Management	✨Added	Records
F52	✨Lab Results / Test Reports Delivery	✨Added	Records
F53	Bi-lingual UI (English & Arabic RTL)	Original	System
F54	✨Offline Mode (Mobile)	✨Added	System
F55	✨Super Admin Panel (Platform-Level Management)	✨Added	System
F56	✨Audit Logs & Activity Tracking	✨Added	System
F57	✨GDPR / Data Privacy & Consent Management	✨Added	System
F58	SEO Optimisation (Patient Web App)	Original	System
F59	✨Dark Mode & Theme Preferences	✨Added	System
F60	✨App Deployment & CI/CD Pipeline	✨Added	System

Authentication & Security

Roles, login, OTP, 2FA, social login

F-01 Email & Password Registration / Login

ORIGINAL Auth

What it does	Patients, doctors, technicians, and facility admins register with email and password. Each role has a different permission set. Supabase Auth manages token issuance, refresh, and session handling automatically.
Platforms	**Mobile · Web · Facility**
How to Achieve	• Use supabase.auth.signUp({ email, password, options: { data: { role } } }) to register a new user — Supabase handles bcrypt hashing and token generation automatically. • Use supabase.auth.signInWithPassword({ email, password }) to log in — returns a session with access_token (JWT) and refresh_token; store in Supabase's built-in session storage. • Supabase auto-refreshes tokens using the refresh_token; call supabase.auth.getSession() on app load to restore an existing session without extra round-trips. • Store the role in user_metadata on sign-up; also mirror it to a public.profiles table via a Supabase Database Function (trigger on auth.users INSERT). • Server-side: in Next.js Route Handlers use createServerClient from @supabase/ssr, read the session from cookies, and call supabase.auth.getUser() to verify the caller. • Role guard: create a middleware.ts using @supabase/ssr that checks user_metadata.role against the required role and returns a 403 JSON response for unauthorised access. • Frontend: call supabase.auth.signOut() to log out — Supabase clears the session cookie automatically; use onAuthStateChange listener to react to session changes across tabs.
Packages / APIs	@supabase/supabase-js · @supabase/ssr · next (Route Handlers) · @supabase/auth-helpers-nextjs

F-02 OTP Phone Verification at Registration

ADDED Auth

What it does	After registering, new patients must verify their phone number via a 6-digit OTP sent by SMS before they can book appointments. This prevents fake accounts and ensures reminders reach real phones.
Platforms	**Mobile · Web**

How to Achieve	• Call supabase.auth.updateUser({ phone }) to attach the phone number — Supabase can send a built-in OTP via SMS if Twilio is configured in the Supabase Auth settings (no custom table needed). • Alternatively, generate a 6-digit OTP with crypto.randomInt(100000, 999999) in a Next.js Route Handler and store OTP hash + expiry in a Supabase otp_records table (user_id, hash, expires_at, attempts). • Send OTP via Twilio SMS from the Route Handler: POST /api/auth/send-otp. • POST /api/auth/verify-otp — compare the submitted OTP against the stored hash; on success call supabase.auth.updateUser({ data: { phone_verified: true } }). • Track attempts: after 5 failures within 15 minutes, insert/update a lock record in otp_records and return 429; unlock automatically when expires_at passes. • Resend endpoint: POST /api/auth/resend-otp — rate-limited with rate-limiter-flexible backed by Supabase or Redis; max 1 resend per 60 seconds. • Mobile: react-native-otp-textinput PIN input; Web: inline 6-box OTP component.
Packages / APIs	twilio · rate-limiter-flexible · @supabase/supabase-js · react-native-otp-textinput

F-03 Two-Factor Authentication (2FA)

✨ADDED Auth

What it does	Facility admins and doctors can optionally enable TOTP-based 2FA (Google Authenticator / Authy). Protects sensitive patient and financial data even if a password is stolen.
Platforms	**Web · Facility**
How to Achieve	• POST /api/auth/2fa/setup — generate a TOTP secret with speakeasy; return a QR code data URI using qrcode; store the encrypted secret temporarily in the user session (not DB yet). • POST /api/auth/2fa/enable — user submits the 6-digit TOTP code; verify with speakeasy.totp.verify(); on success, upsert two_factor_secrets row in Supabase (user_id, encrypted_secret, enabled_at). • POST /api/auth/login — after supabase.auth.signInWithPassword() succeeds, check the profiles table for two_factor_enabled; if true, return { requiresTOTP: true } and a short-lived challenge token instead of a full session. • POST /api/auth/2fa/verify — verify the TOTP code; on success, exchange the challenge token for a full Supabase session using a server-side sign-in. • Generate 10 one-time recovery codes at setup; store bcrypt hashes in a recovery_codes Supabase table; allow login with these if the authenticator app is lost.

	• Frontend: QR code modal; after first scan, prompt for a code to confirm setup; store enabled state in profiles.two_factor_enabled.
Packages / APIs	speakeasy · qrcode · bcrypt · @supabase/supabase-js

F-04 Social Login (Google & Apple)

ADDED Auth

What it does	Patients can sign in with Google or Apple. Reduces friction at registration, especially on mobile.
Platforms	**Mobile · Web**
How to Achieve	• Web: call supabase.auth.signInWithOAuth({ provider: 'google' }) — Supabase handles the OAuth redirect and token exchange natively; no custom backend route needed. • Apple Sign-In on web: supabase.auth.signInWithOAuth({ provider: 'apple' }) — same pattern; configure Apple keys in the Supabase Auth dashboard. • Mobile (React Native): use expo-auth-session to get the provider ID token, then call supabase.auth.signInWithIdToken({ provider: 'google', token }) — Supabase verifies and issues a session. • Apple on mobile: expo-apple-authentication returns an identityToken; pass it to supabase.auth.signInWithIdToken({ provider: 'apple', token }). • On first social sign-in, Supabase auto-creates an auth.users row; trigger a Postgres function to also create a profiles row with role='patient' and the provider identity stored. • Store provider + provider_id in the identities table (Supabase manages this automatically) so the same email can link both social and email-password.
Packages / APIs	expo-auth-session · expo-apple-authentication · @supabase/supabase-js

F-05 Role-Based Access Control (RBAC)

ORIGINAL Auth

What it does	The system has 5 roles: patient, doctor, technician, facility_admin, and super_admin. Each role sees only the features and data they are permitted to access.
Platforms	**All**
How to Achieve	• Store role in auth.users user_metadata and mirror to public.profiles.role (Postgres enum: patient, doctor, technician, facility_admin, super_admin). • Enable Row Level Security (RLS) on every Supabase table; write policies using auth.jwt() ->> 'role' or a helper function get_my_role() that reads from public.profiles.

	• Example RLS policy on appointments: CREATE POLICY 'patients see own' ON appointments FOR SELECT USING (patient_id = auth.uid() OR get_my_role() IN ('doctor','facility_admin','super_admin')). • Next.js middleware.ts: read the Supabase session, extract role from user_metadata, and redirect or return 403 for unauthorised routes before the page renders. • Route Handler guard: create a withRole(handler, ...roles) higher-order function that calls supabase.auth.getUser(), checks the role, and short-circuits with a 403 response. • Frontend: useAuth() hook exposes { user, role, can(action) }; conditionally render nav items and buttons based on role stored in session user_metadata.
Packages / APIs	Supabase RLS policies · @supabase/ssr (middleware) · @supabase/supabase-js

Patient Management

Profile, family accounts, dashboard, document vault

F-06 Patient Profile Creation & Management

ORIGINAL Patient

What it does	Patients set up a personal profile with name, photo, DOB, gender, blood group, phone, and address. Profile is used to pre-fill booking forms and displayed to doctors.
Platforms	**Mobile · Web**
How to Achieve	• Supabase table: profiles (id uuid PK references auth.users, date_of_birth date, gender text, blood_group text, profile_photo_url text, address jsonb, emergency_contact jsonb). • GET /api/patients/me — Route Handler calls supabase.from('profiles').select('*').eq('id', user.id).single(). • PUT /api/patients/me — Route Handler calls supabase.from('profiles').upsert({ id: user.id, ...body }). • Profile photo: upload directly to Supabase Storage bucket 'avatars' using supabase.storage.from('avatars').upload(path, file); save the public URL returned to profiles.profile_photo_url. • Mobile: expo-image-picker to select photo; upload blob to Supabase Storage via the JS client. • Web: react-hook-form with a Supabase Storage upload on file select; optimistic UI update after PUT.
Packages / APIs	@supabase/supabase-js · Supabase Storage · expo-image-picker · react-hook-form

F-07 Family Account Management

ORIGINAL Patient

What it does	A patient can add up to 5 family members under their account and book appointments on behalf of any member. Family members are sub-profiles, not separate accounts.
Platforms	**Mobile · Web**
How to Achieve	• Supabase table: family_members (id uuid PK, patient_id uuid FK → profiles, name text, relation text, dob date, gender text). • POST /api/patients/me/family — Route Handler inserts into family_members; enforce max 5 via a Postgres check constraint or application-level count query. • GET /api/patients/me/family — supabase.from('family_members').select('*').eq('patient_id', user.id).

	• DELETE /api/patients/me/family/[memberId] — supabase.from('family_members').delete().eq('id', memberId).eq('patient_id', user.id). • During appointment booking: include for_family_member_id in the appointment row; RLS ensures only the account owner can create bookings for their family members. • Booking confirmation uses the member's name fetched via a join on family_members.
Packages / APIs	@supabase/supabase-js · Supabase RLS

F-08 Patient Appointment History & Notes

ORIGINAL Patient

What it does	Patients see all past and upcoming appointments. Doctors/admins add clinical notes to completed appointments. Patients see notes (read-only) for their own records.
Platforms	**Mobile · Web · Facility**
How to Achieve	• GET /api/appointments?status=completed — supabase.from('appointments').select('*, doctors(*), facilities(*)').eq('patient_id', user.id).order('slot_date', { ascending: false }). • GET /api/appointments/[id] — fetch single appointment with joined doctor_notes. • Supabase table: appointment_notes (id, appointment_id FK, content text, tags text[], created_by uuid FK, created_at); RLS: insert allowed for doctor/facility_admin roles only. • PUT /api/appointments/[id]/notes — Route Handler inserts into appointment_notes; protected by role check in middleware. • Mobile: History tab with expandable appointment cards showing notes accordion. • Facility app: doctor clicks appointment → text area → Save Notes button calls the Route Handler.
Packages / APIs	@supabase/supabase-js · date-fns

F-09 Document Vault (Upload Prescriptions & Reports)

ADDED Patient

What it does	Patients upload and store health documents — prescriptions, lab reports, X-rays, insurance cards. They can attach any document to a booking for the doctor to review.
Platforms	**Mobile · Web**

How to Achieve	• Supabase table: patient_documents (id, patient_id, name, type text CHECK IN ('prescription','report','imaging','insurance','other'), file_url, file_type, uploaded_at, linked_appointment_id, deleted_at). • POST /api/patients/me/documents — upload file to Supabase Storage bucket 'patient-docs' (max 10 MB, PDF/JPG/PNG); save returned URL to patient_documents table. • GET /api/patients/me/documents — paginated query with optional type filter via Supabase query builder. • DELETE /api/patients/me/documents/[id] — soft delete: set deleted_at = now(); also call supabase.storage.from('patient-docs').remove([path]) to purge the file. • PATCH /api/appointments/[id]/attach-document — update linked_appointment_id on the document row. • Facility app: doctors see attached documents on appointment detail (SELECT with RLS policy: doctor assigned to that appointment can read). • Mobile: Documents tab; tap to preview using expo-web-browser for PDFs.
Packages / APIs	@supabase/supabase-js · Supabase Storage · expo-web-browser · react-pdf

F-10 Medical History Form (Allergies, Conditions, Medications)

✨ADDED Patient

What it does	Patients fill a structured medical history covering allergies, chronic conditions, medications, and surgeries. Displayed to doctors at the top of every appointment.
Platforms	**Mobile · Web · Facility**
How to Achieve	• Supabase table: medical_histories (id, patient_id unique FK, allergies text[], conditions text[], medications jsonb[], surgeries jsonb[], smoking_status text, updated_at). • PUT /api/patients/me/medical-history — upsert into medical_histories using supabase.from(...).upsert({ patient_id: user.id, ...body }). • GET /api/patients/me/medical-history — patient reads own history; RLS: patient_id = auth.uid(). • GET /api/appointments/[id]/patient-medical-history — doctor reads patient history within appointment context; RLS policy: doctor_id on the appointment = auth.uid(). • Mobile/Web: Medical Info section with tag-chip inputs for allergies/conditions; table rows for medications. • Facility app: sticky side-panel in appointment detail view showing medical history.
Packages / APIs	@supabase/supabase-js · react-tag-input

F-11 Favourite Doctors & Facilities

ADDED Patient

What it does	Patients bookmark doctors and facilities for quick rebooking. A Favourites tab shows saved items sorted by most recently added.
Platforms	**Mobile · Web**
How to Achieve	• Supabase table: favourites (id, patient_id FK, target_type text CHECK IN ('doctor','facility'), target_id uuid, created_at); unique constraint on (patient_id, target_type, target_id). • POST /api/patients/me/favourites — toggle: attempt INSERT; if unique violation (23505), DELETE instead (toggle pattern). • GET /api/patients/me/favourites — select with LEFT JOIN to doctors and facilities tables for populated data. • Doctor and facility listing endpoints: LEFT JOIN favourites WHERE patient_id = auth.uid() and return is_favourited boolean per item. • Mobile: heart icon on each card (filled = saved); Favourites tab in bottom nav. • Web: heart icon with optimistic toggle using SWR or React Query mutation.
Packages / APIs	@supabase/supabase-js · Supabase RLS

F-12 Patient Home Dashboard

ADDED Patient

What it does	Personalised home screen showing upcoming appointments, quick-book shortcuts for recent doctors, health reminders, and the AI symptom checker entry point.
Platforms	**Mobile · Web**
How to Achieve	• GET /api/patients/me/dashboard — single Route Handler that runs parallel Supabase queries: upcoming appointments (next 3), recent doctors (last 3 booked), unread notification count. • Use supabase.from('appointments').select('*, doctors(*), facilities(*)').eq('patient_id', uid).gte('slot_date', today).order('slot_date').limit(3). • Cache the dashboard response in Next.js Route Handler using unstable_cache or revalidatePath with a 2-minute TTL to reduce Supabase query load. • Mobile: hero card for next appointment, horizontal scroll for recent doctors, quick-access feature grid. • Web: responsive grid — large appointment card on left, notifications sidebar on right. • Countdown timer on the client using setInterval + date-fns differenceInMinutes.
Packages / APIs	@supabase/supabase-js · next (unstable_cache) · date-fns

□ Doctor & Facility Management

Profiles, availability, reviews, calendars

F-13 Doctor Profile (Bio, Availability, Fees)

ORIGINAL Facility Mgmt

What it does	Each doctor has a public profile showing photo, qualifications, specialty, experience, consultation fee, and weekly availability schedule.
Platforms	**Mobile · Web · Facility**
How to Achieve	• Supabase tables: doctors (id, user_id FK, facility_id FK, specialty, qualifications text[], years_experience, bio, fees jsonb, languages text[], avg_rating numeric); doctor_availability (id, doctor_id FK, day_of_week smallint, slots jsonb[]). • PUT /api/doctors/[id] — facility admin or the doctor themselves can update; RLS enforces ownership. • PUT /api/doctors/[id]/availability — overwrite the weekly schedule; validate slot overlap in a Postgres function; changes reflected in slot-picker immediately via Supabase Realtime. • GET /api/doctors/[id] — public Route Handler, no auth required; supabase.from('doctors').select('*, doctor_availability(*)').eq('id', id).single(). • Mobile: sticky header photo, scrollable bio, star rating display, availability calendar widget, Book CTA. • Web: 2-column layout — photo + info on left, slot picker on right.
Packages / APIs	@supabase/supabase-js · Supabase Realtime · date-fns

F-14 Facility Profile Management

ORIGINAL Facility Mgmt

What it does	Facility admins manage the facility's public listing — name, type, logo, address, working hours, services, and photo gallery.
Platforms	**Web · Facility**
How to Achieve	• Supabase table: facilities (id, name, type text, logo_url, cover_photo_url, address jsonb, working_hours jsonb[], services text[], branches uuid[], rating numeric, location geography(Point,4326)). • PUT /api/facilities/[id] — admin-only Route Handler; validate type enum in a Postgres CHECK constraint. • POST /api/facilities/[id]/photos — upload up to 10 gallery photos to Supabase Storage bucket 'facility-photos'; store URL array in a facility_photos junction table.

	• Use Supabase PostGIS extension for location: INSERT the address coordinates as a geography Point; enables ST_DWithin geo queries for nearby search. • Facility app settings page: tabs for Basic Info, Working Hours, Team, Gallery. • Map pin preview using Leaflet.js after address entry; lat/lng stored in the location column.
Packages / APIs	@supabase/supabase-js · Supabase Storage · Supabase PostGIS · leaflet

F-15 Facility Ratings & Reviews

ORIGINAL Facility Mgmt

What it does	After a completed appointment, patients leave a star rating and written review for the facility and/or doctor. Reviews are public; facility admins can respond but not delete.
Platforms	**Mobile · Web · Facility**
How to Achieve	• Supabase table: reviews (id, patient_id, target_type text, target_id uuid, rating smallint, review_text text, reply jsonb, created_at, is_visible bool DEFAULT true). • POST /api/reviews — verify the patient had a completed appointment with that doctor/facility via a Supabase RPC (Postgres function) before inserting. • GET /api/reviews?target_type=doctor&target_id=[id] — paginated; RLS: is_visible = true for public reads. • PATCH /api/reviews/[id]/reply — facility admin adds a reply; enforce single-reply via a unique partial index on (id) WHERE reply IS NOT NULL. • After INSERT, recalculate avg_rating: Supabase Database Function (trigger) that runs UPDATE doctors SET avg_rating = (SELECT AVG(rating) FROM reviews WHERE target_id = NEW.target_id). • Mobile/Web: star selector + text area on Doctor Profile and Facility Page.
Packages / APIs	@supabase/supabase-js · Supabase Database Functions (triggers) · react-star-rating-component

F-16 Multi-Doctor Team Calendar

ORIGINAL Facility Mgmt

What it does	Facility admins and doctors see a shared calendar showing all doctors' schedules side-by-side. Admins can reassign slots.
Platforms	**Facility**
How to Achieve	• GET /api/facilities/[id]/calendar?date=YYYY-MM-DD&view=day|week — Route Handler fetches all appointments for the facility in the date range, grouped by doctor_id.

	• Query: supabase.from('appointments').select('*, doctors(name), patients(name)').eq('facility_id', id).gte('slot_date', start).lte('slot_date', end). • Frontend: react-big-calendar or FullCalendar in 'resource' view where each column is a doctor. • Each appointment block colour-coded by status (confirmed=green, pending=yellow, cancelled=grey). • Click event → appointment detail modal with patient info, notes, status controls. • Drag-and-drop rescheduling: on drop, call PUT /api/appointments/[id] with new slot (facility_admin role only via RLS). • Subscribe to Supabase Realtime on the appointments table (channel filtered by facility_id) so new walk-in appointments appear instantly without polling.
Packages / APIs	@supabase/supabase-js · Supabase Realtime · react-big-calendar or @fullcalendar/react · date-fns

F-17 Doctor Live Availability Status

✨ADDED Facility Mgmt

What it does	Doctors set real-time status: Available, With Patient, On Break, or Unavailable. Shown to patients on the listing page.
Platforms	**Mobile · Web · Facility**
How to Achieve	• Add status column to doctors table: status text CHECK IN ('available','with_patient','on_break','unavailable') DEFAULT 'unavailable', status_updated_at timestamptz. • PUT /api/doctors/[id]/status — Route Handler updates doctors.status; protected: only the doctor themselves or facility_admin. • Subscribe to Supabase Realtime on the doctors table; all clients in the facility's page receive real-time status updates without polling. • Patient-facing listing: show a coloured dot (green/orange/grey) on each doctor card — populated via the Realtime subscription. • Facility queue screen: doctor status badges with quick toggle buttons for reception staff. • Automatically set status to 'unavailable' using a Supabase Cron Job (pg_cron extension): UPDATE doctors SET status='unavailable' WHERE status_updated_at < now() - interval '15 minutes'.
Packages / APIs	@supabase/supabase-js · Supabase Realtime · Supabase Cron (pg_cron)

F-18 Review Moderation Panel (Facility)

✨ADDED Facility Mgmt

What it does	Facility admins get a panel to read all reviews, flag inappropriate ones, and reply. Protects facilities from defamatory content while keeping the review system trustworthy.
Platforms	**Facility**
How to Achieve	• PATCH /api/reviews/[id]/flag — facility marks a review as inappropriate; sets is_visible=false and inserts a moderation_requests row (review_id, flagged_by, reason, flagged_at). • Super admin reviews flagged content via the admin panel (F-55) and either restores or permanently removes it. • Facility app: Reviews page with tabs: All, Replied, Flagged; inline reply text box below each review. • Supabase Database Function (trigger) on INSERT to moderation_requests: call a Supabase Edge Function that sends an email to the super admin via Resend. • Audit log entry created for every moderation action (F-56).
Packages / APIs	@supabase/supabase-js · Supabase Edge Functions · Resend (email)

Appointments & Scheduling

Booking, queues, check-in, waitlist, calendar sync

F-19 Appointment Booking

ORIGINAL Scheduling

What it does	Patients pick a doctor, date, slot, optionally a family member, confirm, and pay. The system prevents double-booking at the database level.
Platforms	**Mobile · Web · Facility**
How to Achieve	• Supabase table: appointments (id, doctor_id FK, facility_id FK, patient_id FK, slot_date date, slot_start time, slot_end time, status text, type text, payment_status text, for_family_member_id uuid, notes text, created_at). • Unique partial index: CREATE UNIQUE INDEX ON appointments(doctor_id, slot_date, slot_start) WHERE status <> 'cancelled'; this prevents double-booking at the DB level. • POST /api/appointments — Route Handler uses a Supabase RPC (Postgres function) that performs the conflict check and insert atomically inside a BEGIN...COMMIT block, returning 409 on conflict. • After inserting, call a Supabase Edge Function 'send-booking-confirmation' that triggers Twilio SMS + SendGrid email; invoked via supabase.functions.invoke(). • Create a Stripe/Thawani payment session if required; redirect patient to checkout URL. • Mobile booking flow: Search → Doctor → Slot picker → Review & Confirm → Payment → Success.
Packages / APIs	@supabase/supabase-js · Supabase RPC (Postgres transactions) · Supabase Edge Functions · uuid

F-20 Appointment Rescheduling & Cancellation

ORIGINAL Scheduling

What it does	Patients can reschedule or cancel up to a configurable cut-off time. Facilities can cancel any appointment.
Platforms	**Mobile · Web · Facility**
How to Achieve	• PUT /api/appointments/[id] — Route Handler calls a Supabase RPC that atomically frees the old slot and claims the new one in a single Postgres transaction. • DELETE /api/appointments/[id] — set status='cancelled'; check cancellation cut-off with a Postgres function; return 400 'Too late to cancel' if within cut-off window. • Cancellation policy: store cancellation_cutoff_hours in a facility_settings table; the RPC reads it to enforce the cut-off.

	• After cancel/reschedule, invoke Supabase Edge Function 'send-status-notification' to dispatch SMS + push to the patient. • Mobile/Web: Cancel button with confirmation dialog; Reschedule opens slot picker modal. • Freed slot immediately available for new bookings because the unique partial index excludes cancelled rows.
Packages / APIs	@supabase/supabase-js · Supabase RPC · Supabase Edge Functions · date-fns

F-21 One-Tap Follow-Up Rebooking

ORIGINAL Scheduling

What it does	From a completed appointment card, patients tap 'Book Follow-Up' to start a new booking pre-filled with the same doctor and facility. Only need to pick a date.
Platforms	**Mobile**
How to Achieve	• POST /api/appointments/[id]/rebook — Route Handler creates a new appointment row copying doctor_id, facility_id, type from the original; sets follow_up_of = original id. • Mobile: 'Rebook' button on completed appointment card; tapping navigates to Slot Picker screen with doctor_id pre-filled. • No payment is charged at rebook initiation — user still confirms and pays in the normal checkout step. • Doctor sees a 'Follow-Up Visit' badge on appointments where follow_up_of IS NOT NULL.
Packages / APIs	@supabase/supabase-js · react-navigation (navigate with params)

F-22 Queue Management

ORIGINAL Scheduling

What it does	The facility reception dashboard shows a live queue of checked-in patients. Staff can call the next patient and mark patients as Done.
Platforms	**Facility**
How to Achieve	• Supabase table: queue_items (id, facility_id FK, appointment_id FK, patient_name text, is_walkin bool, status text, checked_in_at timestamptz, called_at, done_at, position integer). • POST /api/facilities/[id]/queue/add — inserts a queue_item; position = COUNT(*)+1 for active items. • PATCH /api/queue-items/[id]/call — sets status='called'; PATCH status='done' to remove from active queue. • Subscribe to Supabase Realtime on queue_items filtered by facility_id — all staff screens update instantly without polling.

	• GET /api/facilities/[id]/queue — returns active queue with estimated wait time (active_count × avg_consultation_minutes from facility_settings). • Walk-in add: reception enters patient name + phone → inserts a queue_item with is_walkin=true and a temporary appointment record.
Packages / APIs	@supabase/supabase-js · Supabase Realtime

F-23 Web Check-In

ORIGINAL Scheduling

What it does	Within 30 minutes of their appointment, patients check in via app or website. Notifies the facility and moves the patient into the queue.
Platforms	**Mobile · Web · Facility**
How to Achieve	• POST /api/appointments/[id]/check-in — Route Handler validates appointment status='confirmed' and that current time is within 30 minutes before slot_start; updates status='checked_in' and checked_in_at. • After check-in, insert a queue_item for the patient (calls queue management logic from F-22). • Supabase Realtime broadcast on the appointments table notifies the facility dashboard instantly. • Mobile: 'Check In Now' button appears on appointment card starting 30 minutes before slot (conditional render using date-fns). • Web: same button with countdown 'Check-in opens in X minutes' using setInterval. • Facility app: appointment row shows a 'Checked In' green badge with check-in time.
Packages / APIs	@supabase/supabase-js · Supabase Realtime · date-fns

F-24 Calendar Sync (Google / iCal / Outlook)

ORIGINAL Scheduling

What it does	Patients and doctors can sync appointments to their personal calendar. Supports Google Calendar (OAuth), Apple iCal (.ics), and Microsoft Outlook (.ics).
Platforms	**Mobile · Web · Facility**
How to Achieve	• GET /api/appointments/[id]/ics — Next.js Route Handler returns an .ics file generated with ical-generator; Response headers: Content-Type: text/calendar. • Google Calendar: GET /api/auth/google → redirect to Google OAuth; GET /api/auth/google/callback → save encrypted tokens to user_integrations Supabase table; POST /api/appointments/[id]/google-calendar → insert event via googleapis.

	• Store Google OAuth tokens encrypted in user_integrations (user_id, provider='google_calendar', access_token, refresh_token, expires_at). • Mobile: expo-calendar adds the event directly to the device calendar — no OAuth required. • Web: 'Add to Google Calendar' button (OAuth flow) + 'Download .ics' button (works for Apple/Outlook). • On reschedule or cancellation, update or delete the synced Google Calendar event via the API.
Packages / APIs	ical-generator · googleapis · expo-calendar · @supabase/supabase-js

F-25 Overbooking Control (Race Conditions, Buffer, Walk-ins, Emergencies)

ORIGINAL Scheduling

What it does	Prevents two patients booking the same slot simultaneously. Manages buffer time, reserved walk-in slots, and emergency override slots.
Platforms	**All**
How to Achieve	• Race condition prevention: the unique partial index on appointments(doctor_id, slot_date, slot_start) WHERE status <> 'cancelled' guarantees atomicity at the DB level with no application-level locking needed. • Buffer time: Supabase RPC get_available_slots() subtracts buffer_minutes (from doctor_availability) when generating slot options. • Walk-in slots: mark 1–2 rows per hour in doctor_availability.slots with type='walkin_reserved'; filter these out from the public booking API but expose them in the queue management API (F-22). • Emergency override: facility_admin POSTs to /api/appointments/emergency — Route Handler uses a Supabase RPC that bypasses the unique index check via a special admin bypass flag. • Frontend slot picker: fetches from GET /api/facilities/[id]/available-slots which calls the get_available_slots() RPC to compute availability server-side.
Packages / APIs	@supabase/supabase-js · Supabase RPC (Postgres unique partial index)

F-26 Waitlist Management

✨**ADDED** Scheduling

What it does	When all slots are fully booked, patients join a waitlist. On cancellation, waitlisted patients are automatically notified in order and given 15 minutes to claim the freed slot.
Platforms	**Mobile · Web · Facility**
How to Achieve	• Supabase table: waitlist_entries (id, patient_id, doctor_id, facility_id, preferred_date, position integer, status text CHECK IN ('waiting','offered','expired','booked'), offered_slot jsonb, offered_at timestamptz). • POST /api/waitlist — insert entry with position = COUNT(*)+1 for that doctor+date. • When an appointment is cancelled (F-20), a Supabase Database Function (trigger on appointments UPDATE WHERE status='cancelled') finds the first 'waiting' waitlist entry and updates status='offered' + offered_at. • Supabase Edge Function 'notify-waitlist' is invoked by the trigger: sends push + SMS to the offered patient with a deep link to claim the slot. • POST /api/waitlist/[id]/claim — patient claims within 15 min; creates the appointment; if expired (offered_at + 15 min < now), moves to the next person in queue via another trigger. • Mobile/Web: 'Join Waitlist' button on fully-booked days; Waitlist section in My Appointments showing position.
Packages / APIs	@supabase/supabase-js · Supabase Edge Functions · Supabase Database Triggers · twilio

F-27 Emergency / Walk-In Slot Booking

ADDED Scheduling

What it does	Patients in urgent need can request an emergency visit. Facilities can also book emergency walk-in patients from the reception dashboard.
Platforms	**Mobile · Facility**
How to Achieve	• POST /api/appointments/emergency — skip slot conflict check; insert with type='emergency', status='pending_approval'; this Route Handler uses a Supabase RPC that bypasses the unique index. • Supabase Realtime: INSERT on appointments with type='emergency' broadcasts to the facility's channel — reception dashboard updates instantly. • PATCH /api/appointments/[id]/approve-emergency — facility_admin approves and assigns a time; status changes to 'confirmed'. • Mobile: 'Emergency Visit' option in booking flow with a red CTA; patient writes a reason (required field with min-length check). • Facility app: Emergency requests in a high-priority section at the top of the queue dashboard with an alarm icon. • On approval, Supabase Edge Function sends immediate SMS to the patient with the assigned time.
Packages / APIs	@supabase/supabase-js · Supabase Realtime · Supabase Edge Functions · twilio

F-28 Multi-Location Booking

ORIGINAL Scheduling

What it does	Patients search and book at any branch of a healthcare facility. Filter by location; see branches on a map.
Platforms	**Mobile · Web**
How to Achieve	• Supabase table: branches (id, facility_id FK, address jsonb, location geography(Point,4326), phone, working_hours jsonb[]); PostGIS extension must be enabled. • GET /api/facilities/[id]/branches — supabase.from('branches').select('*').eq('facility_id', id). • GET /api/facilities/nearby?lat=&lng=&radius=5000 — Supabase RPC using ST_DWithin on the location column to find facilities within the specified radius. • GET /api/doctors?facility_id=&branch_id= — filter doctors by facility and optional branch. • Mobile: react-native-maps showing facility pins; tap to see branch details and book; expo-location for the user's current position. • Web: Leaflet.js interactive map with facility/branch pins.
Packages / APIs	@supabase/supabase-js · Supabase PostGIS (ST_DWithin) · expo-location · react-native-maps · leaflet

□ Payments & Billing

Gateway, invoices, refunds, insurance

F-29 Digital Payment Integration (Thawani / Ompay)

ORIGINAL Payments

What it does	Patients pay consultation fees via Thawani or Ompay. Payments processed server-side via webhooks so the client cannot fake success.
Platforms	**Mobile · Web**
How to Achieve	• POST /api/payments/checkout — Next.js Route Handler creates a payment session on Thawani/Ompay API; returns { checkoutUrl, sessionId }. • Supabase table: payments (id, appointment_id FK, patient_id FK, amount numeric, currency text DEFAULT 'OMR', status text, gateway_ref, gateway_response jsonb, created_at). • POST /api/payments/webhook — Thawani posts to this Route Handler; verify HMAC signature with crypto; on success update payments.status and call supabase.from('appointments').update({ status:'confirmed' }). • GET /api/payments/verify/[sessionId] — client-side fallback; reads from Supabase, never from the gateway directly. • Mobile: expo-web-browser opens the checkout URL; deep link returns the user to the app after payment. • Never confirm an appointment based on the client redirect alone — always wait for the webhook to update the DB.
Packages / APIs	axios (Thawani API calls) · crypto (built-in Node) · @supabase/supabase-js · expo-web-browser

F-30 Earnings Dashboard (Facility)

ORIGINAL Payments

What it does	Facility admins see real-time earnings: total revenue by period, per-doctor breakdown, payment method split, and payout history.
Platforms	**Facility**
How to Achieve	• GET /api/facilities/[id]/earnings?period=day|week|month — Supabase RPC using Postgres window functions and date_trunc to aggregate payments by period. • Per-doctor breakdown: GROUP BY doctor_id with SUM(amount). • Payment method split: GROUP BY payment_method with SUM(amount).

	• Render with Recharts: LineChart for revenue over time, BarChart for doctor comparison, PieChart for payment method split. • CSV export: Route Handler runs the same aggregation and streams a CSV response using json2csv. • Payout history: Supabase table payouts (id, facility_id, amount, period_start, period_end, status, processed_at); list in dashboard.
Packages / APIs	@supabase/supabase-js · Supabase RPC · recharts · json2csv

F-31 Invoice & Receipt Generation (PDF)

ADDED Payments

What it does	After successful payment, auto-generates a branded PDF invoice with facility logo, patient/doctor names, service, amount, and tax. Patients receive it by email and can download anytime.
Platforms	**Mobile · Web · Facility**
How to Achieve	• After the webhook confirms payment (F-29), invoke Supabase Edge Function 'generate-invoice'. • Edge Function uses pdfkit to render an invoice template to PDF; uploads the buffer to Supabase Storage bucket 'invoices'; saves the public URL to payments.invoice_url. • GET /api/payments/[id]/invoice — Route Handler returns a redirect to the Supabase Storage public URL. • Send PDF as an email attachment using Resend (attach from the Supabase Storage URL). • Mobile: 'Download Receipt' button opens PDF in expo-web-browser. • Web: 'Download Invoice' button opens payments.invoice_url. • Invoice fields: auto-incremented invoice number, date, facility name/logo/address, patient name, doctor, service, subtotal, tax, total.
Packages / APIs	pdfkit · @supabase/supabase-js · Supabase Storage · Supabase Edge Functions · Resend

F-32 Refund Management

ADDED Payments

What it does	When a paid appointment is cancelled, the system initiates a refund through the gateway. Patients track refund status; facility admins can approve or reject requests.
Platforms	**Mobile · Web · Facility**
How to Achieve	• Refund policy: cancellation by facility → full refund; by patient > 24 h before → full refund; < 24 h → partial (configurable % in facility_settings).

	• POST /api/payments/[id]/refund — Route Handler calculates refund amount from policy, calls Thawani refund API; inserts refunds row (payment_id, amount, reason, status='pending', gateway_refund_ref). • Supabase table: refunds (id, payment_id FK, amount, reason, status, gateway_refund_ref, created_at). • Supabase Edge Function 'poll-refund-status': scheduled via pg_cron every 5 minutes; checks gateway status for pending refunds and updates refunds.status. • Patient notification: Supabase Edge Function sends SMS + push when refund is initiated and again when processed. • Facility app: Refunds tab in earnings section with pending/processed list; admin can override refund amount.
Packages / APIs	axios · @supabase/supabase-js · Supabase Edge Functions · Supabase Cron (pg_cron) · twilio

F-33 Insurance & Coverage Verification

✨ADDED Payments

What it does	Patients add insurance card details. During booking, the system checks if the facility/doctor is covered. If covered, the out-of-pocket amount is adjusted.
Platforms	**Mobile · Web**
How to Achieve	• Add insurance_details jsonb column to profiles table (provider, member_id, policy_number, expiry_date, coverage_type). • Add accepted_insurances text[] column to facilities table. • GET /api/facilities/[id]/insurance-check?provider= — Route Handler queries facilities.accepted_insurances and returns { is_covered, discount_percent, patient_share }. • Adjust checkout amount before creating the payment session in F-29. • Mobile/Web: 'Add Insurance Card' section in profile settings; during booking, show 'Insurance Applied' banner if covered. • Manual verification mode (Phase 1): facility admin manually confirms eligibility in the facility app; auto-verification via insurance APIs as a later phase.
Packages / APIs	@supabase/supabase-js · joi

□ Notifications & Communication

SMS, email, WhatsApp, push, in-app, messaging

F-34 Automated SMS / Email / WhatsApp Reminders

ORIGINAL Notifications

What it does	Automatically sends appointment confirmation, 24 h reminder, 1 h reminder, and cancellation notifications across SMS, email, and WhatsApp. All messages are bilingual.
Platforms	**All**
How to Achieve	• After appointment creation, schedule two reminder jobs via Inngest (or Trigger.dev): delay = appointmentTime - 24h and delay = appointmentTime - 1h. • Supabase table: notification_logs (id, user_id, appointment_id, channel, status, error, sent_at). • Inngest handler calls Twilio SMS, Resend email (with bilingual templates), and WhatsApp Business API (via Twilio) per the patient's notificationPreferences column in profiles. • Create bilingual email templates in Resend: appointment_confirmed, reminder_24h, reminder_1h, appointment_cancelled — each with English and Arabic content. • For WhatsApp: use pre-approved Meta message templates submitted before launch. • Retry logic: Inngest provides built-in retries with exponential back-off; after 3 failures, mark the log record as 'failed'.
Packages / APIs	inngest (or trigger.dev) · twilio · resend · @supabase/supabase-js

F-35 Push Notifications (Mobile & Web)

ORIGINAL Notifications

What it does	Native push on Android (FCM) and iOS (APNs), and browser push on Web. Used for real-time alerts: booking confirmed, queue called, doctor cancelled, test results uploaded.
Platforms	**Mobile · Web**
How to Achieve	• Mobile: expo-notifications; call Notifications.getExpoPushTokenAsync() on app launch; POST token to /api/users/push-token and save to a push_tokens table (user_id, token, platform). • Backend: supabase.from('push_tokens').upsert({ user_id, token }) — upsert deduplicates by token. • Send via Expo Push Notifications API: POST to https://exp.host/--/api/v2/push/send with { to, title, body, data }; call from Supabase Edge Functions.

	• Web push: generate VAPID keys; store browser subscription object in Supabase web_push_subscriptions table; send via the web-push npm package from Edge Functions. • sendPush(userId, { title, body, data }) helper: fetches all tokens for the user and sends to each device. • Handle notification tap on mobile via Notifications.addNotificationResponseReceivedListener; navigate based on data.type. • Remove stale tokens: if Expo returns DeviceNotRegistered, delete that token row from push_tokens.
Packages / APIs	expo-notifications · web-push · @supabase/supabase-js · Supabase Edge Functions

F-36 In-App Notification Centre

✨ADDED Notifications

What it does	A dedicated notifications inbox showing all recent alerts with unread count badge. Users can mark all as read or delete individual notifications.
Platforms	**Mobile · Web**
How to Achieve	• Supabase table: notifications (id, user_id FK, type text, title text, body text, data jsonb, is_read bool DEFAULT false, created_at). • POST /api/notifications (internal, called from Edge Functions) — inserts a notification record. • GET /api/notifications?page=1&limit=20 — supabase.from('notifications').select().eq('user_id', uid).order('created_at', { ascending: false }).range(offset, offset+19). • PATCH /api/notifications/read-all — UPDATE notifications SET is_read=true WHERE user_id=uid. • GET /api/notifications/unread-count — SELECT COUNT(*) FROM notifications WHERE user_id=uid AND is_read=false. • Subscribe to Supabase Realtime on the notifications table filtered by user_id — bell badge updates live without polling. • Mobile: bell icon with red badge; bottom sheet list. Web: bell icon with dropdown; new notifications via Realtime.
Packages / APIs	@supabase/supabase-js · Supabase Realtime

F-37 Doctor–Patient Secure Messaging

✨ADDED Notifications

What it does	After a confirmed appointment, doctors and patients exchange secure messages within the app. Messages are tied to the appointment context.
Platforms	**Mobile · Web · Facility**

How to Achieve	• Supabase tables: conversations (id, appointment_id FK UNIQUE, participants uuid[], last_message text, last_message_at timestamptz, is_active bool); messages (id, conversation_id FK, sender_id FK, sender_role text, content text, sent_at, is_read bool, attachment_url text). • Conversations are auto-created by a Postgres trigger when an appointment status changes to 'confirmed'; is_active is set false 7 days after the appointment date via pg_cron. • Supabase Realtime: subscribe to the messages table filtered by conversation_id — both participants receive new messages instantly. • POST /api/conversations/[id]/messages and GET /api/conversations/[id]/messages?page= for REST history fallback. • File attachments: patients share from their Document Vault (F-09); uploaded to Supabase Storage; URL stored in messages.attachment_url. • Mobile: Chat screen from appointment detail; push notification via F-35 for new messages.
Packages / APIs	@supabase/supabase-js · Supabase Realtime · Supabase Storage

F-38 Broadcast Announcements (Facility to Patients)

✨ADDED Notifications

What it does	Facility admins send announcements to all their patients — closures, new doctors, vaccine availability. Patients receive in-app notification + optional email.
Platforms	**Mobile · Web**
How to Achieve	• POST /api/facilities/[id]/announcements — inserts an announcement record and invokes Supabase Edge Function 'broadcast-announcement'. • Edge Function fetches all relevant patient_ids in batches of 100; creates notification records and sends push via Expo API. • If channels includes 'email': send bulk email via Resend batch send API. • Supabase table: announcements (id, facility_id FK, title, message, channels text[], target_audience text, sent_at, created_by FK). • Rate limit: enforce max 2 broadcasts per day per facility using a Postgres check in the Route Handler. • Patients can mute announcements from a specific facility via a muted_facilities uuid[] column in profiles.
Packages / APIs	@supabase/supabase-js · Supabase Edge Functions · resend · expo push API

AI & Smart Features

Doctor suggestion, symptom checker, scheduling AI, OCR

F-39 AI Doctor Suggestion

ORIGINAL AI

What it does	Based on described symptoms, the AI suggests the most relevant medical specialties and available doctors. Results link directly to the booking flow.
Platforms	**Mobile · Web**
How to Achieve	• POST /api/ai/suggest-doctor — Next.js Route Handler calls OpenAI Chat Completions (gpt-4o-mini) with a system prompt to extract specialties from symptoms; use structured output (response_format: json_object) for guaranteed JSON. • After getting specialties, query Supabase: supabase.from('doctors').select('*').in('specialty', specialties).eq('status','available'). • Return combined response: { ai_reasoning, suggested_specialties, recommended_doctors }. • Rate limit: 5 AI requests per user per hour; check by counting rows in an ai_request_logs table inserted per call (or use Redis via Upstash). • Cache identical symptom queries for 1 hour using Next.js unstable_cache keyed on the symptom string. • Mobile: 'Find the Right Doctor' card → symptom text input → AI result cards with specialty chips + doctor list.
Packages / APIs	openai · @supabase/supabase-js · next (unstable_cache) · Upstash Redis (rate limit)

F-40 In-App Symptom Checker

ORIGINAL AI

What it does	Conversational symptom checker where patients describe symptoms and the AI responds with possible conditions, urgency level, and recommended next steps.
Platforms	**Mobile · Web**
How to Achieve	• POST /api/ai/symptom-check — Route Handler accepts { symptoms, patient_age, patient_gender }. • System prompt instructs the model to assess urgency, list possible conditions in plain language, recommend action, and always include a 'consult a real doctor' disclaimer. • Stream the response using Next.js streaming Route Handler: return new Response(stream) with Content-Type: text/event-stream; pipe the OpenAI stream directly.

	• Mobile: symptom checker screen with chip-based symptom selector + free-text input; animated text stream using useEffect state appending. • Output urgency badge: green (self-care), orange (see doctor soon), red (go to emergency). • Store anonymised queries in a symptom_check_logs Supabase table for future analysis (no PII stored).
Packages / APIs	openai (streaming) · @supabase/supabase-js · next (streaming Route Handlers)

F-41 AI-Based Scheduling Assistant

ORIGINAL AI

What it does	Patients describe needs in natural language and the AI shows matching available slots without manual filtering.
Platforms	**Mobile · Web**
How to Achieve	• POST /api/ai/schedule-assist — Route Handler accepts { query: string, patient_location: { lat, lng } }. • Use OpenAI function calling to extract structured intent: { specialty, preferred_date, preferred_time_range, max_distance, facility_type }. • Once intent is extracted, call Supabase RPC get_available_slots() with the extracted parameters to fetch actual available slots. • Return top 3 matching doctor+slot combinations ranked by relevance and proximity (using PostGIS distance). • Conversational fallback: if AI cannot extract enough info, return { clarifying_question: '...' }. • Mobile: floating chat-style input on the Search screen; results rendered as booking cards.
Packages / APIs	openai (function calling) · @supabase/supabase-js · Supabase RPC

F-42 Prescription OCR Scanner

ADDED AI

What it does	Patients photograph a paper prescription and the AI reads medication names, dosages, and instructions. Extracted data is pre-filled into medical history.
Platforms	**Mobile · Web**
How to Achieve	• POST /api/ai/scan-prescription — Next.js Route Handler accepts a multipart image upload. • Resize/compress server-side using sharp before calling the AI.

	• Call OpenAI Vision API (gpt-4o) with the image and a prompt to extract medications, dosages, frequency, and prescribing doctor as JSON. • Parse JSON response; validate medication names against a basic drug name list in Supabase. • Return extracted medications to frontend; user confirms/edits before saving via PATCH /api/patients/me/medical-history. • Mobile: 'Scan Prescription' button in Document Vault; expo-camera to capture image; loading spinner during OCR.
Packages / APIs	openai (vision) · sharp · expo-camera · @supabase/supabase-js

F-43 AI Post-Visit Health Insights

ADDED AI

What it does	After a completed appointment, AI generates a plain-language summary of doctor's notes for the patient — translating medical jargon into simple explanations.
Platforms	**Mobile · Web**
How to Achieve	• Trigger: Postgres function (trigger on appointment_notes INSERT) invokes Supabase Edge Function 'generate-health-insights'. • Edge Function POSTs to OpenAI with doctor notes + patient age/conditions; prompt: 'Summarise these medical notes in simple language for a patient. Include: what was found, what was prescribed, when to seek urgent care. Max 150 words.' • Store generated summary in appointments.patient_summary; mark with ai_generated=true. • Send push notification to patient: 'Your visit summary is ready' via the push notification service (F-35). • Mobile/Web: 'View Summary' button on completed appointment card. • Include disclaimer banner: 'This summary is AI-generated. Always follow your doctor's direct advice.'
Packages / APIs	openai · @supabase/supabase-js · Supabase Edge Functions · Supabase Database Triggers

□ Telemedicine

Video calls, pre-consultation forms, virtual waiting room

F-44 Video Consultation (Telemedicine)

✨ADDED Telehealth

What it does	Patients book online video appointments. At the scheduled time, both parties join a secure video call room in the app. No third-party Zoom links required.
Platforms	**Mobile · Web · Facility**
How to Achieve	• When appointment type='online' is confirmed, Supabase Edge Function 'create-video-room' calls Daily.co API to create a unique room; stores room_url + room_token on the appointment row. • GET /api/appointments/[id]/join — Route Handler returns the room URL and a time-limited access token (valid in a 2-hour window around appointment time); validates the window server-side. • Mobile: 'Join Call' button 5 minutes before start time; opens Daily.co React Native SDK in-app. • Web: 'Join Call' button opens the video call in an embedded Daily.co Prebuilt UI iframe. • Facility app: doctor sees same Join Call button; split-screen shows patient medical history and document vault. • Post-call: doctor adds consultation notes (F-08 flow); call duration stored via callStartedAt/callEndedAt columns.
Packages / APIs	daily-co SDK · @supabase/supabase-js · Supabase Edge Functions

F-45 Pre-Consultation Form

✨ADDED Telehealth

What it does	Before an appointment, patients fill a short pre-consultation form: chief complaint, symptoms duration, relevant history. Sent to the doctor before they join.
Platforms	**Mobile · Web**
How to Achieve	• Supabase table: pre_consultation_forms (id, appointment_id FK UNIQUE, chief_complaint text, symptoms_duration text, relevant_history text, current_medications text, additional_notes text, submitted_at). • POST /api/appointments/[id]/pre-consultation — validate the form is submitted at least 15 minutes before the appointment (check slot_start in Supabase query). • Facility app: doctor sees the form in appointment detail panel before clicking Join Call.

	• On form submission, Supabase Edge Function sends push notification to the doctor. • Mobile/Web: 'Fill Pre-Consultation Form' prompt appears on the appointment card 2 hours before the slot; completed badge shows after submission.
Packages / APIs	@supabase/supabase-js · Supabase Edge Functions · joi

F-46 Virtual Waiting Room

✨ADDED Telehealth

What it does	For online appointments, patients are placed in a virtual waiting room if the doctor is still with a previous patient. Doctor admits patients when ready.
Platforms	**Mobile · Web**
How to Achieve	• Extend the queue_items table (F-22) with a is_online bool column to support virtual queues for online appointments. • PUT /api/appointments/[id]/virtual-check-in — insert a queue_item with is_online=true; patient is now in the virtual queue. • Doctor dashboard shows the virtual queue; doctor clicks 'Admit Next Patient' — update queue_item status to 'called'. • Supabase Realtime: the patient's app subscribes to their queue_item row; on status change to 'called', automatically navigate to the video call room. • Patient waiting screen: shows queue position, animated waiting graphic, estimated wait time (position × avg_call_duration from facility_settings). • pg_cron job: after 30 minutes in 'waiting' status, set to 'expired' and send a notification: 'Your wait time exceeded.'
Packages / APIs	@supabase/supabase-js · Supabase Realtime · Supabase Cron (pg_cron)

Analytics & Reporting

Booking trends, revenue, staff metrics, PDF export

F-47 Analytics Dashboard (Booking Trends & Patient Demographics)

ORIGINAL Analytics

What it does	Facility admins see data-driven insights: busiest hours, popular doctors, patient demographics, appointment completion vs no-show rates, monthly booking trends.
Platforms	**Facility**
How to Achieve	• GET /api/facilities/[id]/analytics?period=week|month|year — Supabase RPC analytics_summary(facility_id, period) runs Postgres aggregation queries. • Busiest hours: SELECT EXTRACT(HOUR FROM slot_start) AS hour, COUNT(*) GROUP BY hour. • Patient demographics: JOIN profiles on patient_id; GROUP BY gender and age bucket (EXTRACT(YEAR FROM AGE(date_of_birth))). • No-show rate: COUNT WHERE status='no_show' / COUNT WHERE status IN ('confirmed','no_show') * 100. • Top doctors by bookings: GROUP BY doctor_id, SUM(1), ORDER BY count DESC LIMIT 5. • Frontend: Recharts — BarChart (hourly), PieChart (demographics), LineChart (monthly trend), stat tiles. • Cache analytics in Next.js unstable_cache with a 30-minute TTL; 'Refresh' button calls revalidatePath.
Packages / APIs	@supabase/supabase-js · Supabase RPC · recharts · next (unstable_cache)

F-48 Staff Performance Metrics

ADDED Analytics

What it does	Per-doctor performance: total appointments completed, average consultation duration, patient ratings, no-show rate, revenue generated. Used for staff reviews.
Platforms	**Facility**
How to Achieve	• GET /api/facilities/[id]/staff-metrics?doctor_id=&period= — Supabase RPC staff_metrics(facility_id, doctor_id, period) aggregates appointments, payments, and reviews tables. • Total completed: COUNT WHERE doctor_id=X AND status='completed' AND slot_date IN range.

	• Avg consultation duration: AVG(call_ended_at - call_started_at) for online appointments. • Revenue generated: JOIN payments by appointment_id, SUM(amount) GROUP BY doctor_id. • Rating trend: JOIN reviews, AVG(rating) per month GROUP BY DATE_TRUNC('month', created_at). • Facility app: Staff section → click doctor → Performance tab with line charts and KPI tiles. • Export: Supabase Edge Function generates a per-doctor PDF report card using pdfkit.
Packages / APIs	@supabase/supabase-js · Supabase RPC · recharts · pdfkit · Supabase Edge Functions

F-49 Revenue Reports & Payout History

ORIGINAL Analytics

What it does	Detailed financial report: daily/weekly/monthly revenue, breakdown by service type and payment method, outstanding amounts, and payout history.
Platforms	**Facility**
How to Achieve	• GET /api/facilities/[id]/revenue-report?period=month&year=2025 — Supabase RPC revenue_report() aggregates the payments table with DATE_TRUNC partitioning. • Group by date + payment_method + service_type; calculate totals, averages, and growth vs previous period (LAG window function). • Supabase table: payouts (id, facility_id, amount, currency, period_start, period_end, reference_number, status, processed_at). • GET /api/facilities/[id]/payouts — paginated payout history with Supabase query builder. • CSV export: Route Handler runs the same aggregation and pipes through json2csv. • Facility app: Finance tab with summary cards, monthly bar chart, download buttons.
Packages / APIs	@supabase/supabase-js · Supabase RPC · json2csv · recharts

F-50 PDF Report Export

ADDED Analytics

What it does	Facility admins export any analytics view as a professionally formatted PDF. Also generates patient medical history PDFs that can be printed.
Platforms	**Facility**

How to Achieve	• GET /api/facilities/[id]/reports/monthly-summary?month=&year= — Next.js Route Handler triggers a Supabase Edge Function 'generate-report'. • Edge Function fetches aggregated data from Supabase RPC, renders HTML template, uses pdfkit to generate PDF buffer. • Upload generated PDF to Supabase Storage bucket 'reports'; return the public download URL. • Patient medical history PDF: GET /api/patients/[id]/medical-history/pdf — Edge Function generates from profiles + appointment history. • Mobile/Web: 'Export PDF' button on each analytics page; navigates to or downloads the URL.
Packages / APIs	pdfkit · @supabase/supabase-js · Supabase Storage · Supabase Edge Functions

Patient Health Records

Prescriptions, lab results, medical history

F-51 Prescription Management

ADDED Records

What it does	Doctors write digital prescriptions after consultations. Saved to patient record, visible in the app, downloadable as PDF. Patients can share with pharmacies.
Platforms	**Mobile · Web · Facility**
How to Achieve	• Supabase table: prescriptions (id, appointment_id FK, doctor_id FK, patient_id FK, medications jsonb[], instructions text, issued_at). • POST /api/prescriptions — doctor creates after consultation; RLS: only role='doctor' can insert. • GET /api/prescriptions?patient_id=me — patient sees all prescriptions; RLS: patient_id = auth.uid(). • GET /api/prescriptions/[id]/pdf — Supabase Edge Function generates PDF with doctor's digital signature placeholder and facility letterhead using pdfkit; uploads to Supabase Storage. • Share link: GET /api/prescriptions/[id]/share-link — generate a time-limited (24 h) Supabase Storage signed URL using supabase.storage.from('prescriptions').createSignedUrl(path, 86400). • Mobile/Web: Prescriptions tab with medication list + download button per prescription.
Packages / APIs	pdfkit · @supabase/supabase-js · Supabase Storage · nanoid

F-52 Lab Results / Test Reports Delivery

ADDED Records

What it does	Pathology labs upload test results directly to a patient's profile. Patient is notified and can view/download in-app.
Platforms	**Mobile · Web · Facility**
How to Achieve	• POST /api/results — facility (lab/radiology) uploads result file to Supabase Storage bucket 'lab-results'; inserts lab_results row (patient_id, facility_id, test_name, file_url, file_type, notes, is_viewed, uploaded_at). • RLS on lab_results: INSERT allowed for role IN ('facility_admin','technician'); SELECT for own results only (patient_id = auth.uid()). • After INSERT, Supabase Database Trigger invokes Edge Function 'notify-lab-result' which sends push + SMS to the patient. • PATCH /api/results/[id]/viewed — set is_viewed=true when patient opens the result.

	• GET /api/patients/me/results — list results sorted by uploaded_at desc. • Mobile: Results tab with unread badge; tap to preview PDF or image via expo-web-browser.
Packages / APIs	@supabase/supabase-js · Supabase Storage · Supabase Edge Functions · Supabase Database Triggers · expo-web-browser

□ System & Platform Features

RTL, offline, admin panel, GDPR, SEO, CI/CD

F-53 Bi-lingual UI (English & Arabic RTL)

ORIGINAL System

What it does	Every screen supports English and Arabic with RTL layout. Users switch language in settings; choice persists across sessions.
Platforms	**All**
How to Achieve	• Install next-intl in the Next.js app for server and client i18n; create /messages/en.json and /messages/ar.json with all UI strings. • Next.js middleware.ts: detect locale from Accept-Language header or a user preference cookie; redirect to /[locale]/... paths. • Mobile: react-i18next + i18next; persist language choice in AsyncStorage; call I18nManager.forceRTL(true) + RN.restartApp() when switching to Arabic. • Web: on language switch, set document.documentElement.setAttribute('dir', 'rtl') and swap CSS font to IBM Plex Arabic or Cairo. • Use CSS logical properties (margin-inline-start instead of margin-left) so layout auto-flips in RTL. • Navigation arrows: <DirectionalIcon> component with CSS transform: scaleX(-1) in RTL. • Store user's language preference in profiles.language and sync on login so it applies across devices.
Packages / APIs	next-intl · react-i18next · i18next · @supabase/supabase-js

F-54 Offline Mode (Mobile)

ADDED System

What it does	Patient mobile app works in limited offline mode: view cached upcoming appointments, profile, and medical history. Changes sync when connectivity is restored.
Platforms	**Mobile**
How to Achieve	• Use React Query with @tanstack/query-persist-client-core + react-native-mmkv for a persistent query cache that survives app restarts. • useNetInfo hook (@react-native-community/netinfo) to detect connectivity; show 'You are offline — showing cached data' banner when offline. • Offline writes (e.g., editing profile): queue the mutation in an offline queue (AsyncStorage array); process the queue on connectivity restore using the NetInfo listener. • Cache strategy: appointments (5 min), profile (10 min), doctor list (30 min) — configure via React Query staleTime.

	• Mark stale data with a subtle 'Last updated X min ago' label using React Query's dataUpdatedAt.
Packages / APIs	@tanstack/react-query · @tanstack/query-persist-client-core · react-native-mmkv · @react-native-community/netinfo

F-55 Super Admin Panel (Platform-Level Management)

ADDED System

What it does	Separate admin dashboard for the platform owner. Super admins onboard facilities, suspend accounts, view platform-wide analytics, manage moderation, and configure global settings.
Platforms	**Web**
How to Achieve	• Create a Next.js /super-admin route group with middleware that checks role='super_admin' AND verifies the request comes from an allowed IP range. • super_admin role is set directly in Supabase SQL on auth.users user_metadata — never exposed in any public API. • Features: Facility Management (approve/suspend via UPDATE facilities SET status=...), User Management (search profiles, change role), Platform Analytics (aggregate across all facilities), Content Moderation (flagged reviews from F-18), System Config (SMS templates, AI prompts, fee percentages stored in a system_config Supabase table). • All super admin actions create an audit_logs entry (F-56). • All /super-admin/api/* Route Handlers use requireRole('super_admin') via the middleware. • 2FA (F-03) is mandatory for super_admin accounts — enforce in the middleware before rendering any admin page.
Packages / APIs	react-admin or custom Next.js admin UI · @supabase/supabase-js · @supabase/ssr

F-56 Audit Logs & Activity Tracking

ADDED System

What it does	Every sensitive action is recorded in an immutable audit log: who, what, when, from which IP, and what changed. Required for healthcare compliance.
Platforms	**Facility · Super Admin**
How to Achieve	• Supabase table: audit_logs (id, actor_user_id FK, actor_role text, actor_ip text, action text, resource_type text, resource_id uuid, before jsonb, after

jsonb, created_at); enable RLS — INSERT open to service role only, SELECT for super_admin only.

- Enable Postgres row-level immutability: CREATE RULE no_update_audit AS ON UPDATE TO audit_logs DO INSTEAD NOTHING; CREATE RULE no_delete_audit AS ON DELETE TO audit_logs DO INSTEAD NOTHING.
- Insert-only helper: call a Supabase RPC log_audit_event(actor, action, resource, before, after) from Route Handlers for sensitive operations — fire-and-forget (do not await in the main response path).
- Log these events: login, logout, appointment create/update/cancel, payment processed, refund, profile changes, review flagged, user suspended, document accessed.
- GET /api/admin/audit-logs?user_id=&action=&date_from=&date_to= — super admin search with pagination.
- Retention: pg_cron job archives logs older than 2 years to a cold_storage Supabase table (partitioned by year).

Packages / APIs

@supabase/supabase-js · Supabase RLS · Supabase Cron (pg_cron)

F-57 GDPR / Data Privacy & Consent Management

ADDED System

What it does

Patients informed about data collection. They can download all their data, request account deletion, and withdraw marketing consent. Complies with regional healthcare data regulations.

Platforms

All

How to Achieve

- At registration, show a consent screen; store consented=true + consented_at in profiles via supabase.from('profiles').update(...).
- POST /api/users/me/data-export — Route Handler invokes Supabase Edge Function 'export-user-data'; collects all user data (profile, appointments, payments, messages, notes), packages as JSON/ZIP using the archiver npm package, uploads to a private Supabase Storage bucket, and emails a signed download URL via Resend.
- DELETE /api/users/me/account — soft delete: set profiles.status='deletion_pending' + deletion_requested_at; pg_cron job hard-deletes all PII after 30 days using supabase.auth.admin.deleteUser(); keep anonymised analytics records.
- Separate consent flags in profiles.consent_flags jsonb: { marketing, data_sharing, analytics } — users toggle each independently.
- App/Web: Privacy & Data section in settings with 'Download My Data', 'Delete Account', consent toggles.
- Enforce HTTPS-only via Vercel (always-on TLS); add Strict-Transport-Security header via next.config.js headers().

Packages / APIs

@supabase/supabase-js · Supabase Edge Functions · Supabase Cron (pg_cron) · archiver · resend · next (security headers)

F-58 SEO Optimisation (Patient Web App)

ORIGINAL System

What it does	Patient Web App optimised for search engines and AI search. Includes structured data, sitemap, and semantic HTML so new patients discover doctors and facilities through Google.
Platforms	**Web**
How to Achieve	• Use Next.js Metadata API (generateMetadata function) to set unique title, description, and OpenGraph tags on every listing page — no extra package needed. • Doctor profile pages: include JSON-LD structured data (schema.org/Physician) with name, specialty, address, ratingValue via <Script type='application/ld+json'>. • Facility pages: JSON-LD schema.org/MedicalClinic with geo coordinates, opening hours, phone. • Dynamic sitemap: app/sitemap.ts in Next.js returns MetadataRoute.Sitemap with all public doctor and facility profile URLs fetched from Supabase; submit to Google Search Console. • robots.ts: allow all public pages, disallow /dashboard, /admin, /payments. • Next.js App Router provides SSR by default for all pages — no extra configuration needed for SEO-friendly rendering. • Core Web Vitals: next/image for lazy-loading + WebP conversion; code-split with dynamic().
Packages / APIs	next (Metadata API, App Router SSR, next/image) · @supabase/supabase-js

F-59 Dark Mode & Theme Preferences

ADDED System

What it does	Users toggle between Light, Dark, and System themes. Preference persists and applies instantly across all screens without a reload.
Platforms	**Mobile · Web**
How to Achieve	• Web: CSS custom properties for all colours; toggle data-theme='dark' on <html> via JS; use next-themes library for easy SSR-safe theme toggling; store preference in a cookie (not localStorage) for SSR consistency. • Mobile: React Native's Appearance API + ThemeContext provides theme colours throughout the app; persist preference with AsyncStorage. • useTheme() hook returns { theme, toggleTheme, colors } — use colors.background, colors.text everywhere (never hardcode). • Avoid pure black (#000) in dark mode — use #121212 background and #E0E0E0 text.

	• Sync theme preference to profiles.theme_preference column via PATCH /api/users/me/preferences so it applies on login from a new device.
Packages / APIs	next-themes · AsyncStorage (mobile) · CSS custom properties · @supabase/supabase-js

F-60 App Deployment & CI/CD Pipeline

ADDED System

What it does	Automated build and deployment pipeline: code merged to main is tested, built, and deployed. Mobile apps submitted to app stores via automated EAS builds.
Platforms	**All**
How to Achieve	• Next.js frontend + API Routes: deploy to Vercel; connect the GitHub repo in Vercel dashboard — every push to main auto-deploys; every PR gets a preview URL. • Supabase: use Supabase CLI with supabase db push to run migrations; store migration files in /supabase/migrations and apply via GitHub Actions on merge to main. • GitHub Actions workflow: on push to main → run Jest/Vitest tests → supabase db push (migrations) → Vercel deployment (triggered automatically via Vercel GitHub integration). • Mobile: Expo Application Services (EAS Build) — eas build --platform android/ios; on PR merge, trigger EAS Submit to publish to Play Store + App Store automatically. • Environment separation: dev, staging, production Supabase projects; separate Vercel environments with env variable groups. • Health check: GET /api/health — Route Handler returns { status: 'ok', db: 'connected', version } checked by UptimeRobot.
Packages / APIs	Vercel · Supabase CLI · GitHub Actions · EAS CLI · @expo/eas-cli